



dekay / DavisRFM69 ▾



Code

Issues 4

Pull requests 2

Actions

Projects

Wiki

Message Protocol

Edit

New page

[Jump to bottom](#)

Chris Matteri edited this page on Sep 18, 2015 · [10 revisions](#)

This page describes the message protocol used by the Davis Weather wireless Integrated Sensor Suite (ISS) to communicate its readings back to the console. The starting point for reverse engineering the protocol is the output from the STRMON command when connected to the console with an LVTTL serial connection. This connection [is discussed in detail here](#).

Note: since that post was written, Davis issued a firmware update to lock out this kind of connection along with disabling third-party data loggers. If your console is equipped with Firmware 3.0 or greater, a simple USB to Serial Adapter isn't going to work. JoeBean on WXForum.net [wrote an epic post](#) that summarizes the reverse engineering effort by many others into one concise writeup detailing how to build your own data logger.

Let's use the following STRMON snippet as an example.

```
0 = 60
1 = 6
2 = d3
3 = ff
4 = c0
5 = 0
6 = 78
7 = 75
```



The eight bytes come from the ISS in this order. However, each byte comes in from the ISS with least significant bit first. The bit order has to be flipped before we can work with it. All values in the example above are in hex.

Byte 0: This is a header. The upper nibble is the sensor the data is from, as follows. Some of these messages are Vantage Vue specific.

```
2 = Supercap voltage (Vue only)
3 = ?
4 = UV Index
5 = Rain rate
6 = Solar radiation
7 = Solar Cell output (Vue only)
8 = Temperature
9 = Wind gust
a = Humidity
e = Rain
```



The lowest three bits in the low order nibble is the transmitter ID, [set via dipswitches inside the unit](#).

Bit 3 in the low order nibble of Byte 0 indicates if the transmitter battery is low. The bit is

set to zero if the battery is OK, [but is apparently only set if the transmitter needs to run off the battery and not the solar-charged supercap.](#)

Byte 1: Wind speed in mph. Wind speed is updated every transmission. Simple.

Byte 2: Wind direction from 1 to 360 degrees. Wind direction is updated every transmission. The wind reading is contained in a single byte that limits the maximum value to 255. Davis says that 0 indicates it can't get a reading, so you'd never see wind straight out of the North [unless your wind vane is broken.](#)

Wind direction is treated differently between the VP2 and the Vue because of the different types of sensors used between the two units. The VP2 uses a potentiometer with a significant dead zone around North. No values are reported between 352 and 8 degrees inclusive. These values correspond to received byte values of 255 and 1 respectively. So for a VP2...

cmatteri: This code conflicts with the previous two paragraphs in that it treats 0 as a valid reading. With my VP2 ISS, I often get 0 readings when the wind is coming from the north, so I'm pretty sure 0 is a valid reading.

```
if (Byte2 == 0)
    windDirection = 360;
else
    windDirection = 9 + Byte2 * 342.0 / 255.0;
```



The Vue uses a Hall Effect sensor and this dead zone is significantly reduced. [This is the code](#) for calculating the wind speed for the Vue.

```
if (Byte2 == 0)
    windDirection = 360;
else
    windDirection = (Byte2 * 1.40625) + .3;
```



Byte 6: High byte of the 16 bit CRC (0x78 in our example above)

Byte 7: Low byte of the 16 bit CRC (0x75 in our example above)

The CRC is the same as that on the serial interface and is documented in the Davis "VantageSerialProtocolDocs_v230.pdf" document. It's name is CRC-16-CCITT. The first six bytes can be run through the calculation and checked against the seventh and eighth bytes. Alternatively, all eight bytes can be run through the calculation and the result will be zero if the CRC is valid. My code uses [this CRC algorithm](#).

Bytes 3 - 5: Depend on the sensor being read at the time. Need to work through these. This is what is known now.

Message 2: Supercap voltage (Vue only)

Bytes 3 and 4 are for reporting the Supercap voltage. The Supercap is a "super" capacitor that stores excess energy from the ISS solar cell during the day, which is then used to help power the console at night. This design is very effective in extending the life of the non-rechargeable lithium CR2032 battery in the outdoor unit. Both the VP2 ISS and the Vue have a Supercap, but only the Vue reports its status. [Dario has determined](#) that the value is reported over a range of 3V to 8V and is calculated as follows:

```
voltage = ((Byte3 * 4) + ((Byte4 && 0xC0) / 64)) / 100
```



Message 4: UV Index

Bytes 3 and 4 are for UV Index. The first byte is MSB and the second LSB. The lower nibble of the 4th byte is always 5, so they only use the first three nibbles. A value of FF in the third byte indicates that no sensor is present.

The UV index is calculated as follows as discussed [here](#) and [here](#).

```
UVIndex = ((Byte3 << 8) + Byte4) >> 6) / 50.0
```



Message 5: Rain Rate

Bytes 3 and 4 contain the rain rate information. The rate is actually the time in seconds between rain bucket tips in the ISS. The rain rate is calculated from the bucket tip rate and the size of the bucket (0.01" of rain for units sold in North America). More information [here](#) and [here](#).

TBD: Is the rain bucket size different in different parts of the world, or is it the same and just calculated differently?

TODO: write this up in actual code and (preferably) convert to the console's native units of inches per hour

```
no rain      if Byte3 == 0xFF

rain intensity:

light rain   if (Byte4 && 0x40) == 0
strong rain  if (Byte4 && 0x40) == 0x40

light rain:
```



```
time between clicks[s] = ((Byte4 && 0x30) / 16 * 250) + Byte3
rainrate [mm/h] = 720 / (((Byte4 && 0x30) / 16 * 250) + Byte3)
```

strong rain:

```
time between clicks[s] = (((Byte4 && 0x30) / 16 * 250) + Byte3) / 16
rainrate [mm/h] = 11520 / (((Byte4 && 0x30) / 16 * 250) + Byte3)
```

Message 6: Solar Radiation

Bytes 3 and 4 are solar radiation. The first byte is MSB and the second LSB. The lower nibble of the 4th byte is again always 5, so they only use the first three nibbles. A value of FF in the third byte indicates that no sensor is present. See [here](#) and [here](#) for further discussion.

```
Solar radiation = (((Byte3 << 8) + Byte4) >> 6) * 1.757936
```



Message 8: Temperature

Byte 3 and 4 are temperature. The first byte is MSB and the second LSB. The value is signed with 0x0000 representing 0F. This reading in the old version of the ISS was taken from an analog sensor and measured by an A/D. The newer ISS uses a digital sensor but still represents the data in the same way. 160 counts (0xa0) represents 1 degree F. A message of

```
80 04 70 0f 99 00 91 11
```



represents temperature as 0x0f99, or 3993 decimal. Divide 3993 by 160 to get the console reading of 25.0F

```
tempF = ((Byte3 * 256 + Byte4) / 160)
```



Message 9: Wind Gust Speed

gust speed Msg-ID 0x9 (every 50 seconds): This message transmits the maximum wind speed during the last 10 minutes (it appears to be the maximum in the last ten message 9 intervals, for an exact time of $\text{packet_period} * \text{message_9_period} * 10 = (40 + \text{id}) / 16 * 20 * 10$). The units are in miles per hour, in-keeping with the Davis convention of using Imperial units as their base and converting in the console as required.

It is likely that the ISS only measures the wind once for each packet. This would imply that the value in message 9 will never be higher than the wind value in byte 1 of some packet. Message 9, due to its long period, has poorer precision than the wind speed in byte 0, and it does not report the direction of the wind gust. Thus, it is best to use message 9 as a backup in case a packet reporting a gust was missed.

The upper nibble of byte 5 contains an index ranging from 0 to 9 that indicates which of the last ten message 9 intervals the gust occurred in. If there is a tie between two or more intervals, the index is set to the most recent one. This information can be used to only count each gust once, or perhaps more importantly, to not count a gust when it was recorded in byte 1 of some packet. See [here](#) (specifically calculate_wind_gust) for an implementation in a Weewx driver.

Message 9 includes an index

```
gust = Byte3
gust_index = Byte5 >> 4
```



Message a: Humidity

Humidity is represented as two bytes in Byte 3 and Byte 4 as a ten bit value. Bits 5 and 4 in Byte 4 are the two most significant bits. Byte 3 is the low order byte. The ten bit value is then 10x the humidity value displayed on the console. The function of the four low order bits in Byte 3 that cause the apparent jitter are not known.

```
humidity = (((Byte4 >> 4) << 8) + Byte3) / 10.0
```



Here is an example using an actual message from my console.

```
a0 06 52 83 38 00 5a c8
```



The corresponding humidity value is then

```
((0x38 >> 4) << 8) + 0x83 = 131 + 768 = 899 = 89.9% Relative Humidity
```



The displayed humidity at the time was 90%. The console rounds the value. [More here.](#)

Message e:

Rain is in Byte 3. It is a running total of bucket tips that wraps back around to 0 eventually from the ISS. It is up to the console to keep track of changes in this byte. Only bits 0

through 6 of byte 3 are used, so the counter will overflow after 0x7F (127).

The example below is bound to confuse: the leading value is the elapsed time since data collection started (in seconds), all bytes have been converted to decimal, and the last two CRC bytes have been stripped off. A tip of the rain bucket causes the value the ISS is sending from a steady value of 40 to a new value of 41.

```
2426.3,224,16,33,40,1,0
2436.6,224,11,36,40,1,0
2446.8,224,9,29,41,2,0
2457.1,224,10,29,41,3,0
```



Message f:

[Soil Moisture Station Protocol](#)

Protocol Summary

To summarize:

- Byte 0 is a header.
- Byte 1 always represents wind speed
- Byte 2 always represents the wind direction
- Bytes 3-5 will carry other data according to the header in Byte 0
- Bytes 6 and 7 always represents the checksum with high byte first

Repeaters

Davis actually uses 10 byte packets, but if you are not attempting to receive transmissions from repeaters, it is safe to ignore the last two bytes, which are both set to 0xff in packets from non-repeater transmitters.

In packets from repeaters, bytes 8 and 9 contain data regarding the repeaters whose purpose is not yet known (though it doesn't need to be known to receive data from repeaters). The CRC in bytes 6 and 7 is calculated on bytes (1, 2, 3, 4, 5, 8, 9), as opposed to bytes (1, 2, 3, 4, 5) for non-repeater packets.

Miscellaneous Notes

From my notes to help work the rest of this out:

Signed values from the weather station console are two's complement, least significant byte first. Note that pressure is measured by the console and not the ISS, so don't expect

it to appear in the STRMON output.

Update rates below are from Davis' specs and manuals. Data sizes and signing are as per the loop command and are what one might expect out of STRMON but not always. I noted above that wind direction via STRMON is actually one byte unsigned. There may be other exceptions.

- Outside temp: 10 seconds in 10th of a degree F, two bytes signed (message e in STRMON, bytes 3 and 4).
- Winds speed: 2.5 seconds, one byte unsigned (Byte 1 in STRMON, always)
- Wind direction: 2.5 seconds, two bytes unsigned from 1 to 360 (one byte via STRMON, Byte 2 always)
- Outside humidity: 50 seconds in percent, one byte unsigned (message a in STRMON, bytes 3 and 4)
- Rain: 10 seconds. This is in counts of 0.01 inches.
- Pressure: in Hg/1000, two bytes unsigned (Rate???)
- Leaf Wetness: 40 seconds
- Soil Moisture: 40 seconds
- Solar radiation: 50 seconds
- UV: 50 seconds
- Soil Moisture: 62.5 seconds

I think all other outdoor related values are calculated in the console.

40 50 60 80 90 a0 e0



The rates they show up at are (with a transmitter ID of 1, i.e. one packet every 2.5 s):

- 40 shows either every 47.5 or 50 seconds
- 50 shows every 10 seconds
- 60 shows every 50 seconds
- 80 shows every 10 seconds
- 90 shows either 45, 47.5, or 50 seconds
- a0 shows alternately every 40 seconds and 10 seconds (interesting!)
- e0 shows every 10 seconds

[Kobuki has suggested](#) that the message types for the VP2 ISS follow the following fixed order, which is mostly consistent with the above timings. It's unlikely that messages 4 and 9 can show up at rates other than every 20th packet, as that would mess up the timings for 8, e, and 5.

8, e, 5, 4,
8, e, 5, 9,
8, e, 5, a,
8, e, 5, a,
8, e, 5, 6



+ Add a custom footer

▼ Pages 5

Find a page or section...

▼ Home

▼ Additional Resources

▼ Hardware Hacks

▼ Message Protocol

Message 2: Supercap voltage (Vue only)

Message 4: UV Index

Message 5: Rain Rate

Message 6: Solar Radiation

Message 8: Temperature

Message 9: Wind Gust Speed

Message a: Humidity

Message e:

Message f:

Protocol Summary

Repeaters

Miscellaneous Notes

▼ RF Protocol

+ Add a custom sidebar

Clone this wiki locally

<https://github.com/dekay/DavisRFM69.wiki.git>

